AURA Stage 8.1 — Actual Kaggle Pilot: Evidence Audit and Decision

Audit date: 25 October 2026
Input: `AURA_STAGE8_1_OUTPUTS.zip` supplied after running Kaggle notebook v4.
Archive SHA256: `5afbd5486a893a742b27a57639cae2a645107bc8ed5ede7ada4baf9f30bb454b`
Stage 7 implementation SHA256: `210d7c963aba647c436cba495da5e924062951082e1352fadd9ea8469cc94ff8` (matches the saved Stage 7 source ZIP).
Conclusion: PARTIAL_LOCAL_ONLY_ANALYSIS — genuine local model-action pilot succeeded and was audited; native AppWorld task execution was blocked before starting. **Stage 8.1 external validation is not complete.**

1. What was run (and what was not)

- Kaggle runtime: Python 3.13.15, NVIDIA Tesla T4, CUDA-enabled Torch 2.11.0+cu128, Transformers 4.57.6.
- Agent: **Qwen/Qwen2.5-3B-Instruct**, using actual model-generated tool outputs in the controlled local student-marks task. This was **inference**, not training.
- AURA: existing Stage 7 conservative verifier with saved task contract, probe catalogue, initial/final snapshots, tool trace, observations and per-episode certificates. The on-Kaggle regression receipt reports **186 passed in 32.49 s**; the Stage 7 source ZIP hash matches the local reference archive.
- Track A: **4/4 local model episodes audited**, **8 actual tool events** across 4 episodes. Frozen contract: correct student 101 marks=85, exactly one record, report exists and matches DB, no unauthorized *persistent* changes within the declared frame, validation before update.
- Track B: **0/3 AppWorld tasks executed; 0 official evaluations; 0 no-action controls; 0 AURA-on-AppWorld certificates.** The notebook reported `PARTIAL_LOCAL_ONLY_ANALYSIS`.
- The four deterministic preflight cases matched their expected verdicts (`PASS`, `FAIL`, `FAIL`, `UNCERTAIN`), but those are **scripted engineering smoke tests**, not four additional real-agent episodes.

2. Episode-by-episode results

- **01 Direct, no fault:** 3 tool calls; student 101 ends at 85; report correct. **AURA PASS**, independent local reference success.
- **02 Procedural, no fault:** 0 tool calls; student stays at 70; no report. **AURA FAIL**, independent local reference failure.
- **03 Update-tool no-op:** 3 tool calls; student stays at 70 but report says 85. **AURA FAIL**, independent local reference failure.
- **04 Student-state probe denied:** 2 tool calls; student ends at 85; report correct, but validation was skipped. **AURA FAIL**, independent local reference failure.

Observed model-action outcomes: 1 success, 3 failures, 0 cases where the full local AURA verdict was UNCERTAIN. The scripted readback-denial preflight *did* yield UNCERTAIN, but that does not establish the rate of abstention on genuine model trajectories.

A model-format failure is not a completed workflow

The procedural episode generated text containing multiple concatenated JSON commands or unstructured completion claims. The frozen parser accepted **none of the ten turns**. There was no state update or report to verify. This matters for later experiments: do not describe this episode as an agent that attempted the tool calls and whose tools malfunctioned. It is an **action formatting / agent interface failure**.

Direct evidence beats an unrelated unknown

In the denied-readback episode, the `student_state` observation was `unavailable` (`permission_denied`); `marks_correct` and `exact_record_count` were unknown. However, the independent trace contained an update operation without a preceding validated eligibility check. AURA correctly returned `FAIL` with `known_violation_dominates_unknown`. This does **not** test the case where all known requirements succeed and the *only* missing evidence is denied readback.

Important caveat on the "silent" fault

The injected no-op tool returned `status: success` **and** a warning that the tool response was not independently verified. AURA's detection comes from the real final-state mismatch. For publication, the injected tool-return conditions should be precisely specified rather than called fully silent without qualification.

3. Integrity and evidence audit

An independent re-audit of the supplied archive found:

- **57 ZIP members:** 55 SHA256-manifested research files, plus the manifest and README.
- ZIP CRC: valid. Manifest entry names, byte lengths and **55/55 SHA256 values**: valid. No path-traversal entries or duplicate filenames.
- Across the four episodes, all saved before/after snapshot digests, trace-event digests and observation payload digests recomputed correctly.
- Certificate references match included observation identifiers; certificate contract hashes match the exported contract.
- Each real episode has an independent local reference, AURA certificate, raw model turns, observations, snapshots, tool trace, policy comparison, run manifest and non-interference receipt. Report content is included when a report exists and its hash matches the recorded file manifest.
- Each episode's `noninterference_receipt.json` records `verifier_modified_task_state: false`, and the kernel receipt reports unchanged Torch, Transformers, JSONSchema, Pytest and Pydantic versions. These are **scoped checks** of that execution, not a universal security proof.
- The ZIP does **not** include unnecessary model weights, virtual environments, all installation logs, hidden AppWorld evaluation programs or raw AppWorld databases. The absence of AppWorld run artifacts is explained by the failed setup, **not** an exporter omission.

Archive SHA256 checks establish self-consistency with the accompanying manifest, **not** external cryptographic authenticity of the experiment.

4. CERA and baseline policies — actual tiny pilot

The saved per-policy receipts allow a descriptive comparison for these *four local episodes*. The following values add measured probe execution and planner-selection times, and **exclude the common initial/final snapshot capture cost**.

Policy	Mean selected probes	Mean (probe + planner) time, ms
Fixed order	2.50	0.457
Cost greedy	2.25	0.470
CERA	**2.25**	**1.912**
Exhaustive	4.00	0.733

CERA generally performed fewer inspections than exhaustive, but planning overhead dominates these tiny local tasks. On this dataset, **CERA was not faster** than fixed or cost-greedy. With just four episodes, tiny runtimes and shared small fixtures, **no statistical or general performance superiority should be claimed**. CERA's nominal normalized inspection cost results from earlier simulation stages must not be substituted for these observed wall-clock measurements.

5. External benchmark blocker — AppWorld

The official AppWorld phase failed at virtual environment creation:

```
CalledProcessError(1, [.../aura_stage8_1_appworld_venv/bin/python3, -m, ensurepip, --upgrade,
--default-pip])
```

On the Kaggle runtime, the `venv.EnvBuilder(with_pip=True)` route attempted to seed pip via `ensurepip` and failed. **No AppWorld environment, official task result, agent trajectory or native API log was obtained.** The v4 notebook correctly abstained from assigning external scores.

Prepared repair (v5): replace the `ensurepip`-dependent environment creation with `uv venv` and install AppWorld via `uv pip install --python <isolated_python> appworld`. `uv` can install into a Python environment without pip being present inside it. This exact isolation mechanism was tested here with a self-contained local wheel, and the v5 notebook passes structural/code checks, 186 inherited tests, a mocked full pipeline, certificate-corruption fail-closed tests, and simulated installer-error reporting.

Still unverified: a genuine AppWorld package install/download/agent run. This sandbox cannot resolve `pypi.org`; an attempt to download AppWorld returned a DNS error. Therefore the v5 repair is a **locally mechanism-tested fix**, not a guarantee that the next Kaggle native benchmark phase will succeed.

6. Limitations worth fixing before a research paper

1. **No external AURA certificates.** Native AppWorld scoring (once it runs) remains separate from independent AURA verification. We still need a translated AppWorld contract, independent evidence adapter and held-out trajectory evaluation.
2. **Very small, authored task family.** Four local model runs are a feasibility pilot, not an accuracy benchmark or a statistically powered evaluation.
3. **Tool trace arguments are not fully preserved.** The saved `TraceRecorder` events show tool names and outputs but have empty argument arrays; the model's arguments are separately retained in `agent_turns.json`. Before making general workflow-order claims, bind trace events to exact tool arguments and targets.
4. **A simulated probe denial is not a total database outage.** The student-state read probe was forced unavailable, while other probes still used trusted captured state. Name the scope of the denial precisely.
5. **Reference and verifier inspect the same local sandbox.** Their implementations differ, but they share the environment and assumptions, and do not constitute independent external ground truth.
6. **Final-state snapshots miss temporary reverted writes.** The closed-world persistent frame only covers the declared database table and reports directory. Out-of-scope external effects are not certified.
7. **The reference flag `no_extra_files` conflates missing report and extra files.** The actual reference verdict is unaffected here, but rename or separate this diagnostic in a future revised experiment.
8. **Do not silently modify prompt/decoder rules after this pilot.** The procedural episode provides genuine information about weak structured output. Future constrained decoding or function-calling fixes must be labeled as a **new, separate experiment**.

7. Final recommendation

Do not declare Stage 8.1 finished. Save the supplied ZIP unchanged as a successful **Track A / partial Stage 8.1 record**, then run the AppWorld-fix v5 notebook to collect native external trajectories and official scores. The v5 notebook runs a fresh model trial with a distinct experiment name; do not combine the old and new local episodes into a single unchanged experiment. Once AppWorld evidence exists, build a task-specific AURA adapter and evaluate AURA against independent external agent outcomes.

Final status: Track A validated within scope; output ZIP complete and integrity-checked; Track B unexecuted due to `ensurepip`; no independent external AURA evaluation yet.